

```
Python
# Project Documentation Code
project_title = "Data Analytics Final Project: Air Quality Analysis & Monitoring"
domain = "Environmental & Health Analytics"
objective = "Perform exploratory data analysis (EDA) on air quality metrics across monitoring
stations and states, cleaning missing values using multi-level median imputation, and
identifying spatial and pollutant-specific trends."
```

# Air Quality Analysis & Monitoring Document

## 1. Project Overview

This project focuses on analyzing air quality data across various monitoring stations and states using **Pandas** for data cleaning and pre-processing, along with **Matplotlib** and **Seaborn** for statistical visualization. The core objective is to identify spatial and pollutant-specific trends, assess regional variations, and handle missing observations using a robust hierarchical imputation strategy.

## 2. Tools & Environment

- **Pandas:** Data loading, manipulation, deduplication, and hierarchical median imputation.
- **NumPy:** Numerical computations and mathematical transformations.
- **Matplotlib:** Base visualization engine and figure layout configuration.
- **Seaborn:** Advanced statistical visualizations, heatmaps, scatter plots, and density estimates.

## 3. Dataset Description

- **Domain:** Environmental & Health Analytics
- **Total Observations:** 3,353 records
- **Total Features:** 11 columns
- **Key Columns:**
  - **country, state, city, station:** Spatial metadata identifying location hierarchy.
  - **last\_update:** Datetime stamp of the reading.
  - **latitude, longitude:** Geographic coordinates.
  - **pollutant\_id:** Type of air pollutant monitored (e.g., \$PM\_{2.5}\$, \$PM\_{10}\$, \$NO\_2\$, \$SO\_2\$, \$CO\$, \$NH\_3\$).
  - **pollutant\_min, pollutant\_max, pollutant\_avg:** Observed minimum, maximum, and average pollutant concentration values.

## 4. Execution Steps & Methodology

### Step 1: Data Ingestion & Setup

The pipeline imports necessary dependencies, applies publication-grade visual themes (**whitegrid**, **muted** palette), and dynamically loads raw records from standard CSV or Excel files.

```
Python
import os
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns

# Global theme parameters
sns.set_theme(style="whitegrid", palette="muted")
plt.rcParams["font.sans-serif"] = "DejaVu Sans"
plt.rcParams["figure.dpi"] = 100

# Data Ingestion
file_location = r"C:\Users\kevin\Downloads\3b01bcb8-0b14-4abf-b6f2-c1bfd384ba69.csv"
if file_location.endswith(".xlsx") or file_location.endswith(".xls"):
    df_raw = pd.read_excel(file_location)
else:
    df_raw = pd.read_csv(file_location)
```

## Step 2: Data Pre-processing & Cleaning

1. **Deduplication:** Identified and dropped duplicate rows across all attributes.
2. **Datetime Formatting:** Converted **last\_update** values to standardized Pandas **datetime** format using **%d-%m-%Y %H:%M:%S**.
3. **Hierarchical Median Imputation:** Addressed missing metrics (**pollutant\_min**, **pollutant\_max**, **pollutant\_avg**) through a sequential fallback mechanism:
  - **Level 1:** Group-level median by (**station**, **pollutant\_id**).
  - **Level 2:** Group-level median by (**state**, **pollutant\_id**).
  - **Level 3:** National global median for remaining missing entries.

```
Python
def preprocess_air_quality_data(df):
    df_clean = df.copy()

    # 1. Deduplication
    if df_clean.duplicated().sum() > 0:
        df_clean = df_clean.drop_duplicates()

    # 2. Datetime parsing
    df_clean["last_update"] = pd.to_datetime(
        df_clean["last_update"], format="%d-%m-%Y %H:%M:%S", errors="coerce"
    )

    # 3. Hierarchical median imputation
    metrics = ["pollutant_min", "pollutant_max", "pollutant_avg"]
```

```

for metric in metrics:
    df_clean[metric] = df_clean.groupby(["station", "pollutant_id"])[metric].transform(
        lambda x: x.fillna(x.median())
    )
    df_clean[metric] = df_clean.groupby(["state", "pollutant_id"])[metric].transform(
        lambda x: x.fillna(x.median())
    )
    df_clean[metric] = df_clean[metric].fillna(df_clean[metric].median())

return df_clean

```

```
cleaned_df = preprocess_air_quality_data(df_raw)
```

### Step 3: Exploratory Data Analysis & Visualization

Visualizations were constructed to evaluate distributions, cross-pollutant comparisons, spatial concentration matrices, and numerical correlation structures.

Python

```
# --- Visualization 6: Mean Level by Pollutant Type ---
```

```

plt.figure(figsize=(8, 4.5))
avg_pollutants = cleaned_df.groupby("pollutant_id")["pollutant_avg"].mean().reset_index()
sns.barplot(
    data=avg_pollutants,
    x="pollutant_id",
    y="pollutant_avg",
    hue="pollutant_id",
    palette="rocket",
    legend=False,
)
plt.title("6. Mean Concentration Level by Pollutant Type", fontsize=12, fontweight="bold")
plt.xlabel("Pollutant Type")
plt.ylabel("Mean Pollutant Average")
plt.tight_layout()
plt.show()

```

```
# --- Visualization 7: Mean Max Concentration ---
```

```

plt.figure(figsize=(8, 4.5))
sns.barplot(
    data=cleaned_df,
    x="pollutant_id",
    y="pollutant_max",
    hue="pollutant_id",
    palette="Blues_d",
    legend=False,
)
plt.title("7. Mean Maximum Concentration by Pollutant Type", fontsize=12, fontweight="bold")
plt.xlabel("Pollutant Type")

```

```

plt.ylabel("Mean Pollutant Max Level")
plt.tight_layout()
plt.show()

# --- Visualization 8: Correlation Heatmap ---
plt.figure(figsize=(6.5, 4.5))
num_cols = ["pollutant_min", "pollutant_max", "pollutant_avg"]
corr_matrix = cleaned_df[num_cols].corr()
sns.heatmap(corr_matrix, annot=True, fmt=".2f", cmap="coolwarm", vmin=-1, vmax=1)
plt.title("8. Correlation Matrix of Raw Numerical Metrics", fontsize=12, fontweight="bold")
plt.tight_layout()
plt.show()

# --- Visualization 9: Min vs Max Scatter Plot ---
plt.figure(figsize=(7.5, 5))
sns.scatterplot(
    data=cleaned_df,
    x="pollutant_min",
    y="pollutant_max",
    hue="pollutant_id",
    alpha=0.6,
)
plt.title("9. Minimum vs. Maximum Pollutant Readings", fontsize=12, fontweight="bold")
plt.xlabel("Pollutant Minimum")
plt.ylabel("Pollutant Maximum")
plt.tight_layout()
plt.show()

# --- Visualization 10: State vs Pollutant Matrix ---
plt.figure(figsize=(9, 5))
top_5_states = cleaned_df["state"].value_counts().head(8).index
pivot_df = cleaned_df[cleaned_df["state"].isin(top_5_states)].pivot_table(
    index="state", columns="pollutant_id", values="pollutant_avg", aggfunc="mean"
)
sns.heatmap(pivot_df, annot=True, fmt=".1f", cmap="YlGnBu", cbar_kws={"label": "Mean Level"})
plt.title("10. State vs Pollutant Average Concentration Matrix", fontsize=12, fontweight="bold")
plt.xlabel("Pollutant Type")
plt.ylabel("State")
plt.tight_layout()
plt.show()

```

## 5. Key Findings & Strategic Insights

- Strong Linear Metric Relationship:** Correlation matrices and scatter plots display strong positive linear alignment among `pollutant_min`, `pollutant_max`, and

`pollutant_avg`, pointing to consistent baseline ambient levels during measurement cycles.

- **Regional Disparities:** Aggregations across geographic states indicate that heavily industrialized clusters systematically register elevated average pollutant concentrations relative to national benchmarks.
- **Particulate Matter Skewness:** Distribution profiling demonstrates that particulate matter ( $PM_{2.5}$  and  $PM_{10}$ ) exhibits pronounced right-skewness and higher average intensity compared to gaseous compounds ( $SO_2$ ,  $NO_2$ ,  $NH_3$ ).
- **Monitoring Station Density:** Operational reporting frequency is concentrated within major metropolitan stations, revealing higher sensor density and urban data capture.
- **Imputation Integrity:** The hierarchical fallback strategy (Station  $\rightarrow$  State  $\rightarrow$  Global) maintained localized distribution variance without skewing national metric medians.

## 6. Project Artifacts

- `Cleaned_Air_Quality_Final.csv`: The processed dataset with full imputation and normalized records.
- `air_quality_analysis.py`: Python module containing processing logic and visualization code.